ASSIGNMENT #1

Performance Benchmarking of Cryptographic Mechanisms

Filipa Melo Leite & Luana Letra Gibbels

If we want to use cryptographic mechanisms it is essential to understand how the different algorithms work, how they perform, in which aspects they distinguish one from another and in which they are similar. In the following assignment, we will analyze the AES, RSA, and SHA algorithms and compare their encryption and decryption time for different file sizes.

Main componets of the code

```python
def encrypt_aes(plaintext, key, iv):
    cipher = Cipher(algorithms.AES(key), modes.CFB(iv), backend=default_backend())
    encryptor = cipher.encryptor()
    encrypted = encryptor.update(plaintext) + encryptor.finalize()
    return encrypted

def decrypt_aes(encrypted, key, iv):
    cipher = Cipher(algorithms.AES(key), modes.CFB(iv), backend=default_backend())
    decryptor = cipher.decryptor()
    decrypted = decryptor.update(encrypted) + decryptor.finalize()
    return decrypted

def encrypt_decrypt_aes(file_size):
    key = os.urandom(32)
    iv = os.urandom(16)

    with open(f'size{file_size}.txt', 'rb') as f:
        plaintext = f.read()
    encrypted = encrypt_aes(plaintext, key, iv)

    decryption_time = timeit.timeit(lambda: decrypt_aes(encrypted, key, iv), number=1)
    encryption_time = timeit.timeit(lambda: encrypt_aes(plaintext, key, iv), number=1)

    return encryption_time, decryption_time

def create_lists():
    file_sizes_aes = [8, 64, 512, 4096, 32768, 262144, 2097152]

    aes_encryption_times = []
    aes_decryption_times = []

    for file_size in file_sizes_aes:
        save_text_to_file(generate_random_text(file_size), f'size{file_size}.txt')
        encryption_time, decryption_time = encrypt_decrypt_aes(file_size)
        aes_encryption_times.append(encryption_time*1000000)
        aes_decryption_times.append(decryption_time*1000000)
    return aes_encryption_times, aes_decryption_times
```

AES: The code snipped above shows a version of the AES algorithm. In order to encrypt and decrypt files of differnet sizes we created a function '*def create_list*' that focuses on passing the different file sizes succesively to '*def encrypt_decrypt_aes*' and saving the return values in two saperated lists. The file size gets passed to '*def encrypt_decrypt_aes*' which will generate a new key of 256-bits and a new inicialization vector each time its called. It will read the file of the according size and save the content in '*plaintext*' and another string is created with the encrypted version of the plaintext. With those strings and the implementation of '*timeit*' we are now able to measure the time the functions for encryption and decryption take in a particular case. For each process we have a separate function '*def encrypt_aes*' and '*def decrypt_aes*' which both will receive the according string, a key and the inicialization vector and return or the plaintext or the ciphertext. The results will be returned to '*def create_list*' and multiplied by 1000000 to save the content in microseconds. In this case the focus relies on the time therefor the encrypted and decrypted strings will not be written back or saved into a file.

```python
def generate_rsa_key_pair():
    private_key = rsa.generate_private_key(
        public_exponent=65537,
        key_size=2048,
        backend=default_backend()
    )
    public_key = private_key.public_key()
    return private_key, public_key

private_key, public_key = generate_rsa_key_pair()


def encrypt_rsa_file(plaintext, public_key):

    encrypted = public_key.encrypt(
        plaintext,
        padding.OAEP(
            mgf=padding.MGF1(algorithm=hashes.SHA256()),
            algorithm=hashes.SHA256(),
            label=None
        )
    )
    return encrypted

def decrypt_rsa_file(encrypted, private_key):
    decrypted = private_key.decrypt(
        encrypted,
        padding.OAEP(
            mgf=padding.MGF1(algorithm=hashes.SHA256()),
            algorithm=hashes.SHA256(),
            label=None
        )
    )
    return decrypted

def encrypt_decrypt_rsa(file_size):

    with open(f'size{file_size}.txt', 'rb') as f:
        plaintext = f.read()

    encrypted = encrypt_rsa_file(plaintext, public_key)
    decryption_time = timeit.timeit(lambda:decrypt_rsa_file(encrypted,private_key), number= 1)
    encryption_time = timeit.timeit(lambda:encrypt_rsa_file(plaintext, public_key), number=1)

    return encryption_time, decryption_time

def rsa_lists():
    file_rsa = [2,4,8,16,32,64,128]

    rsa_encryption_time = []
    rsa_decryption_time = []

    for file_size in file_rsa:
        save_text_to_file(generate_random_text(file_size), f'size{file_size}.txt')
        encryption_time, decryption_time = encrypt_decrypt_rsa(file_size)
        rsa_encryption_time.append(encryption_time*1000000)
        rsa_decryption_time.append(decryption_time*1000000)
    return rsa_encryption_time,rsa_decryption_time
```

RSA: This code snipped is similar to the code created for the AES algorithm. First, we generated a public key and a private key, both of size 2048-bits, which will be maintained for all the different encryption/decryption processes. To be able to calculate the encryption/decryption time successively for each file size without needing to call the function multiple times we created '*def rsa_list*' which will pass the file sizes to '*def encrypt_decrypt_rsa*'. Here we will open the file of the according length and save its content into a temporary variable. We pass the plaintext to '*def encrypt_rsa*' to get a string with the ciphertext. Further, we use the timeit module to calculate how long '*def encrypt_rsa_file'* takes to encrypt the plaintext and '*def decrypt_rsa_file*' takes to decrypt the ciphertext. In this case, we need to consider that both of the functions use padding, in RSA padding ensures that the plaintext is properly formatted before encryption and that it can be properly decrypted later, but it has an impact on our performance and therefore we might notice some affects in the time measured. After the time is measured for both functions, it will be returned to '*def rsa_list*' where a list for encryption and a list for decryption will save the measured time.

```python
def sha256(plaintext):
    digest = hashes.Hash(hashes.SHA256())
    digest.update(bytes(plaintext,"ascii"))
    digest.finalize()

def sha_list():
    sha_sizes= [8,64,512,4096,32768,262144,2097152]
    sha_hash_generation_time = []

    for file_size in sha_sizes:
        save_text_to_file(generate_random_text(file_size), f'size{file_size}.txt')

        with open(f'size{file_size}.txt', 'r') as f:
            plaintext = f.read()
        execution_time = timeit.timeit(lambda: sha256(plaintext), number=1)
        sha_hash_generation_time.append(execution_time*1000000)
    return sha_hash_generation_time
```

SHA: The Sha algorithm works differently than the algorithms for RSA and AES. It is a one-way algorithm, once it produces a hash digest it is not possible to generate the plaintext the hash code represents. As above the function '*def sha_list*' serves to calculate and save the execution time of the algorithm. For each file size, we will open the according file and save it temporary to '*plaintext*'. This

variable will be passed to '*def sha256*' where a specific hash code is generated. Notice that each plaintext generates a specific fixed-sized hash code, so every time that plaintext is passed to the function the same hash code will be produced. In the next step, we want to combine the hash code and the plaintext, to do so the plaintext needs to be converted to binary and afterward it can be combined. The last step is to finalize the hash value and produce the final digest. As we are only interested in the time the function takes to execute we do not return any value.

Time of cryptographic operations/algorithms

size (bits)	*AES encryption*	*AES decryption*	*RSA encryption*	*RSA decryption*	*SHA digest generation*
2	/	/	47.538	1540.563	/
4	/	/	46.733	1419.660	/
8	22.603	23.418	48.114	1419.118	12.199
16	/	/	46.399	1416.970	/
32	/	/	46.783	1419.368	/
64	18.188	18.992	46.599	1420.648	6.441
128	/	/	45.355	1414.271	/
512	19.047	20.100	/	/	6.518
4096	25.109	24.897	/	/	8.128
32768	72.123	68.990	/	/	24.320
262144	46.7589	417.524	/	/	154.710
2097152	3809.431	3405.309	/	/	1338.284

Experimental Setup:

System Information:

Operating System: macOS-14.3.1-arm64-arm-64bit

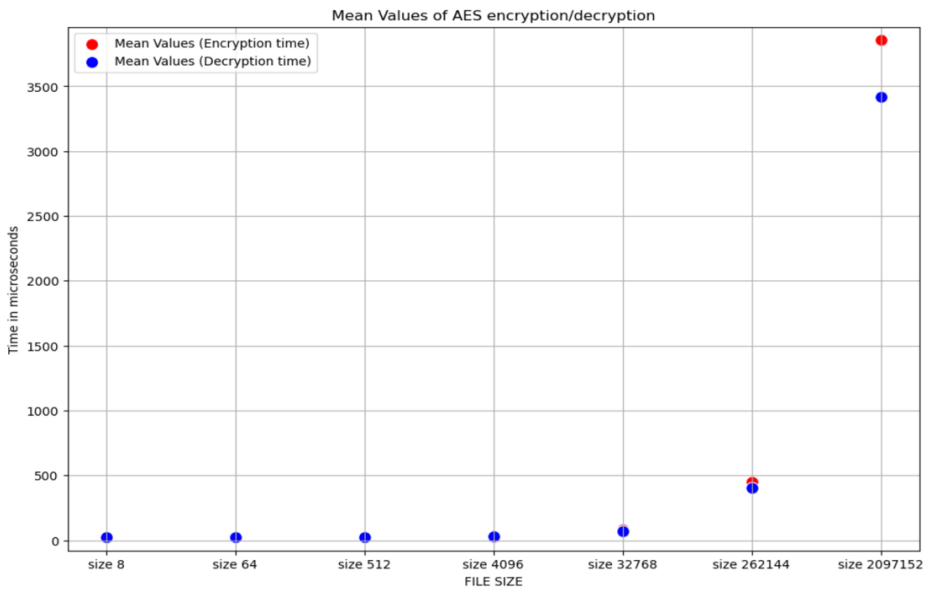
Processor Architecture: 64bit

Processor Information: arm

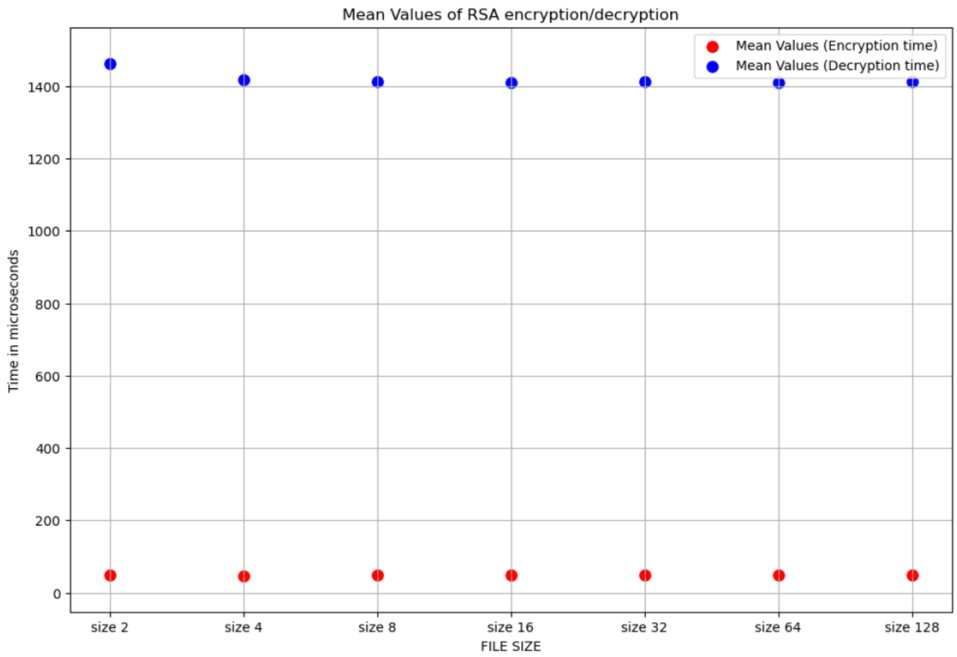
Python Version: 3.11.7 (main, Dec 15 2023, 12:09:56) [Clang 14.0.6]

After testing the algorithm several times over the same files, we noticed that the execution time for the encryption/decryption is not constant and changes slightly each time we called the function. To obtain statistically significant results we used the pandas module to create a dataframe in which we saved the execution time of each algorithm for encryption and decryption. After adjusting the code to calculate the execution time a hundred times we then proceeded to calculate the mean values of each file size and execution time.

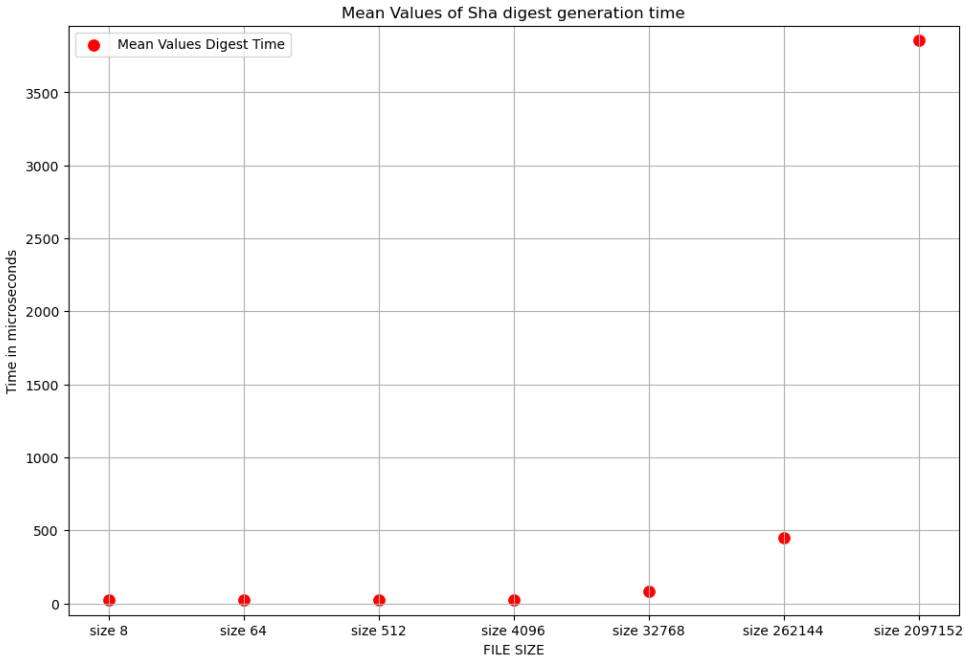
To verify what happens if we generate a text file each before calculating the execution time. We implemented the code accordingly so it would generate the desired information. Upon analysing the uptained values the difference after each iteration was minimal and compared to fixed-size files similar. Further, we will focus on the execution time and their behavior for fixed-size files.



The plot above shows both, encryption time and decryption time of the AES algorithm. Notice that for the sizes 8, 64,512, 4096 and 32768-bits it is not possible to see the mean values for the encryption time, this is due to the fact that both processes have the same mean value for the identical file sizes. At the files with bit representation below 4049-bit the value differences are significantly small, all around 0 microseconds per process, and in this representation impossible to distinguish. Starting at file size 32768-bits we can notice an increase in the mean time values the algorithm takes to encrypt/decrypt and that the time for encryption tends to increase more than the time for decryption with each larger file size we measured.



The plot for the RSA algorithm illustrates that the decryption time is much higher than the encryption time. We can also see that for all sizes the mean values are all around the same time with slight changes. Only for the decryption of size 2, it seems that it takes slightly longer than for any other size. All in all, the encryption time seems to stay consistent over the analyzed file size, so does the decryption time.



The plot for the SHA algorithm illustrates the mean values the algorithm takes to produce a digest. For the sizes 8, 64, 512 and 4096 we can see that they all stay around 0 microseconds and seem not to vary. After that, the time starts to increase more with each larger file size we measured.

Comparison between AES encryption and RSA encryption

If we look closer to the AES encryption and the RSA encryption we can see that the AES algorithm takes much less time for small-sized files, below 32768, than the RSA algorithm takes for files with less than 128 bits. As we can not predict the behavior of the encryption of larger file size for the RSA algorithm, we can say that the performance of files with less than 128 bits encryption is significantly better with the AES algorithm.

Comparison between AES encryption and SHA gigest generation

Comparing the the plot for the AES algorithm and the SHA algorithm we can see that they behave similarly. The first measured file sizes tend to stay around 0 microseconds per execution. Only after file size 32768-bits both plots start to show an increase in time with each larger file size measured. If we consider the mean values given in the table we can see that the SHA digest is produced in half of the time the AES encryption takes to produce the ciphertext. To conclude, the SHA algorithm has a better performance than the AES algorithm to produce an encryption of the file sizes.

Comparison between RSA encryption decryption

The mean values obtained from the RSA encryption clearly show that the decryption process takes much longer than the encryption for each file size. For each file size measured, we can see that the algorithm takes more or less the same time, for encryption the values stay around 46 microseconds per execution and for decryption around 1435 microseconds per execution. RSA shows a much better performance for encryption than for decryption.